Common Platform Specification

One shared architecture behind all three agents — harness, gateway, orchestration, observability and failure behaviour, defined once and inherited.

Why this exists

Video Agent, Sales Agent and SQL Agent are three products on one stack. Everything common — the agent harness, model routing, error taxonomy, logging and trace model — is specified here and referenced by the product PRDs rather than repeated. Where a product PRD is silent, this document governs.

Canonical stack

Language / API	Python 3.12 · FastAPI (async)
LLM gateway	LiteLLM proxy — single egress for every model call
Models	Gemini · OpenAI · Claude, referenced only by logical alias
Orchestration	LangGraph — every agent is a compiled StateGraph
Observability	Langfuse — traces, generations, scores, prompt registry
Relational	PostgreSQL 16 — system of record, RLS per tenant
Vector	pgvector (default) or MongoDB Atlas, behind one protocol
Cache	Redis 7 — cache, locks, rate limits, idempotency, progress

Agent harness & loop engine

The harness owns context, tools, budgets and termination. The model is a component inside it, never the controller.

```
observe → think → act → evaluate → repeat | terminate
| escalate
```

TERMINATION CONDITION	OUTCOME
Evaluator satisfied	SUCCESS
Budget exhausted (iterations, time, tokens, USD)	PARTIAL — best-so-far, flagged `degraded`
Same failure signature twice	FAILED_NO_PROGRESS — stop immediately
Non-retryable error / human trigger	FAILED / ESCALATED

Model routing

Code never names a provider. Aliases resolve at the gateway, so swapping models is a config change with zero code diff.

`reasoning-high`	Planning, SQL generation, critique
`reasoning-fast`	Routing, classification, extraction
`realtime-voice`	Low-latency conversational turns
`embed-default`	All embeddings
`vision-default`	Frame inspection, continuity QC

Failure behaviour

- **Retry** — exponential backoff + jitter, retryable errors only, max 3
- **Fallback** — alternate model within the alias group
- **Circuit break** — per dependency, 5 failures in 30s
- **Degrade** — cached, stale or partial result, always flagged
- **Fail honestly** — what happened, what was preserved, what to do next

Every error response carries a stable code and the `trace_id`, so support opens the exact Langfuse trace instantly.

Observability

Trace = one unit of work. Spans = graph nodes. Generations = LLM calls with model, tokens, cost and prompt version. Logs are JSON with the Langfuse `trace_id`, so any log line joins to its trace.

Never logged: credentials, raw PII, full media payloads, row-level query results.

Non-negotiables

- Checkpoint after every node — crashes resume, never restart
- Hard budget caps on iterations, wall-clock, tokens and dollars
- Idempotency keys on every work-creating POST
- Untrusted content (crawled, retrieved, tool output) never issues instructions
- CI gates on eval regression >3% and cost regression >20%

Rollout

Migrations are expand/contract and applied before deploy. Every new agent behaviour sits behind a feature flag. Model and prompt changes go to 10% of traffic first and are promoted only after their Langfuse scores hold against the incumbent.